Design Patterns in SmartBank

SmartBank — Software Engineering project documentation

This document catalogs the design patterns intentionally applied in the SmartBank codebase. Each entry describes the pattern, the motivation, and points to the concrete implementation.

---

1. Application Factory (Creational)

Where: `app/__init__.py` — `create_app(test_config=None)`

Problem: we need a single, reusable way to construct a Flask app that works for both production (`run.py`) and tests (where each test case wants its own isolated database file).

Solution: expose a factory function that builds and configures a Flask instance on demand. Tests override `DATABASE` and `TESTING` via `test_config`; the production entry point uses defaults.

```
def create_app(test_config=None):
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_mapping(...)
    if test_config:
        app.config.update(test_config)
    ...
    return app
```

Benefit: no global app state, trivial to parameterize for tests, clean lifecycle (DB init + admin seeding happen inside `app_context()`).

---

2. Repository / Data-Access Module (Structural)

Where: `app/models.py`

Problem: route handlers should not build SQL strings directly — this leaks persistence concerns into the controller layer and duplicates code.

Solution: all database access is funneled through `app/models.py` functions (`register_user`, `deposit`, `withdraw`, `transfer`, `transaction_history`, ...). Routes only call these functions; they never touch `sqlite3` directly.

Benefit: business rules (e.g. *"amount must be positive"*, *"insufficient funds"*) live in exactly one place and are covered by fast unit tests without spinning up the HTTP stack.

---

3. Template Method (via UML Transaction hierarchy)

Where: `app/models.py` — `deposit`, `withdraw`, `transfer` + `_log_tx`, `_notify`

Problem: the UML class diagram defined a `Transaction` base class with `DepositTransaction`, `WithdrawTransaction`, `TransferTransaction` subclasses. All three types share a common outline: `validate → mutate balance → persist a transaction row → notify → commit`.

Solution: the three public functions follow the same skeleton and delegate the variable steps (balance mutation, which account to notify) to small helpers `_log_tx` and `_notify`. This is a procedural flavor of the Template Method — the "template" is the ordered sequence of steps, and each concrete operation fills in the variable parts.

```
def deposit(db, account_number, amount, description="Deposit"):
    # 1. validate  2. mutate  3. log  4. notify  5. commit
    ...

def withdraw(db, account_number, amount, description="Withdraw"):
    # same skeleton, different mutation + notification text
    ...

def transfer(db, src, dst, amount, description=""):
    # same skeleton, two balance updates + two notifications
    ...
```

Benefit: adding a new operation (e.g. `currency_exchange`) means writing one function that follows the same pattern — the rest of the system (logging, notifications) comes for free.

---

4. Observer / Publish–Subscribe (Behavioral)

Where: `app/models.py` — `_notify` helper + `notifications` table

Problem: when a transaction completes, both affected users need to be informed. Hard-coding notification logic inside each transaction function would be brittle.

Solution: each domain operation publishes a notification event into the `notifications` table via `_notify(db, user_id, type_, message)`. Subscribers (currently the `/notifications` page and the dashboard "Recent notifications" card) read from that table. It's a simple database-backed pub/sub — the producer never knows or cares who is listening.

Why the UML calls it `NotificationManager`: the class diagram has a `NotificationManager` with `sendNotification(customerID, message)`. In the code that role is currently collapsed into the `_notify` helper; if we add channels (email, push, SMS) they would be registered as extra subscribers without changing the producer sites.

---

5. Decorator (Behavioral)

Where: `app/routes.py` — `login_required`, `admin_required`

Problem: every protected route would otherwise start with a copy of `if "user_id" not in session: return redirect(...)` — duplication that is easy to forget.

Solution: two view decorators wrap the view function. They intercept the request, enforce the precondition, and either call through or short-circuit to a redirect / 403.

```
def login_required(view):
    @wraps(view)
    def wrapped(*a, **kw):
        if "user_id" not in session:
            return redirect(url_for("bank.login"))
        return view(*a, **kw)
    return wrapped
```

Benefit: authorization is declarative (`@login_required` / `@admin_required`) and visible at the top of every protected route.

---

6. Front Controller + Blueprint (Structural)

Where: `app/routes.py` — `bp = Blueprint("bank", __name__)`

Problem: we need a single entry point that dispatches incoming HTTP requests to the right handler, and we want to keep route registration separate from the application factory.

Solution: all routes are collected on a Flask **Blueprint**, which is registered once inside `create_app`. Flask's URL map acts as the front controller, routing URLs to handlers.

Benefit: the blueprint can be mounted under a prefix, tested in isolation, or split into multiple blueprints (`auth_bp`, `admin_bp`) without touching the factory.

---

7. Singleton-per-request (Idiomatic Flask)

Where: `app/db.py` — `get_db()` / `close_db()`

Problem: we want exactly one SQLite connection per HTTP request, reused across all `models.*` calls within that request, and closed when the request ends.

Solution: `get_db()` lazily creates a connection on Flask's request-scoped `g` object and returns the same one on subsequent calls within that request. `close_db` is registered as a teardown callback.

Benefit: a per-request "singleton" avoids both the cost of opening many connections and the correctness hazard of sharing one global connection across threads.

---

8. Active Row (lightweight Active Record)

Where: `app/db.py` — `g.db.row_factory = sqlite3.Row`

Problem: raw tuples returned by `sqlite3.Cursor.fetchone()` force positional access (`row[0]`, `row[1]`) that couples callers to column order.

Solution: setting `row_factory = sqlite3.Row` gives dict-like rows that can be accessed by column name (`row["balance"]`). It is a thin, read-only flavor of the Active Record pattern — no ORM, no lazy loading, just ergonomic row access.

Patterns we chose not to use

- **Full ORM (SQLAlchemy):** overkill for a ~5-table coursework project; `sqlite3` + parameterized queries already give us safety and clarity. - **Strategy (for validation rules):** the current validation is simple enough to be expressed inline. Promoting each rule to a strategy would add indirection without benefit today. - **Event-sourcing / CQRS:** the transaction log (`transactions` table) already gives us an audit trail; a full event-sourced model would be unjustified complexity at this scale.

These are worth revisiting if the project grows (NFR-05 Scalability).